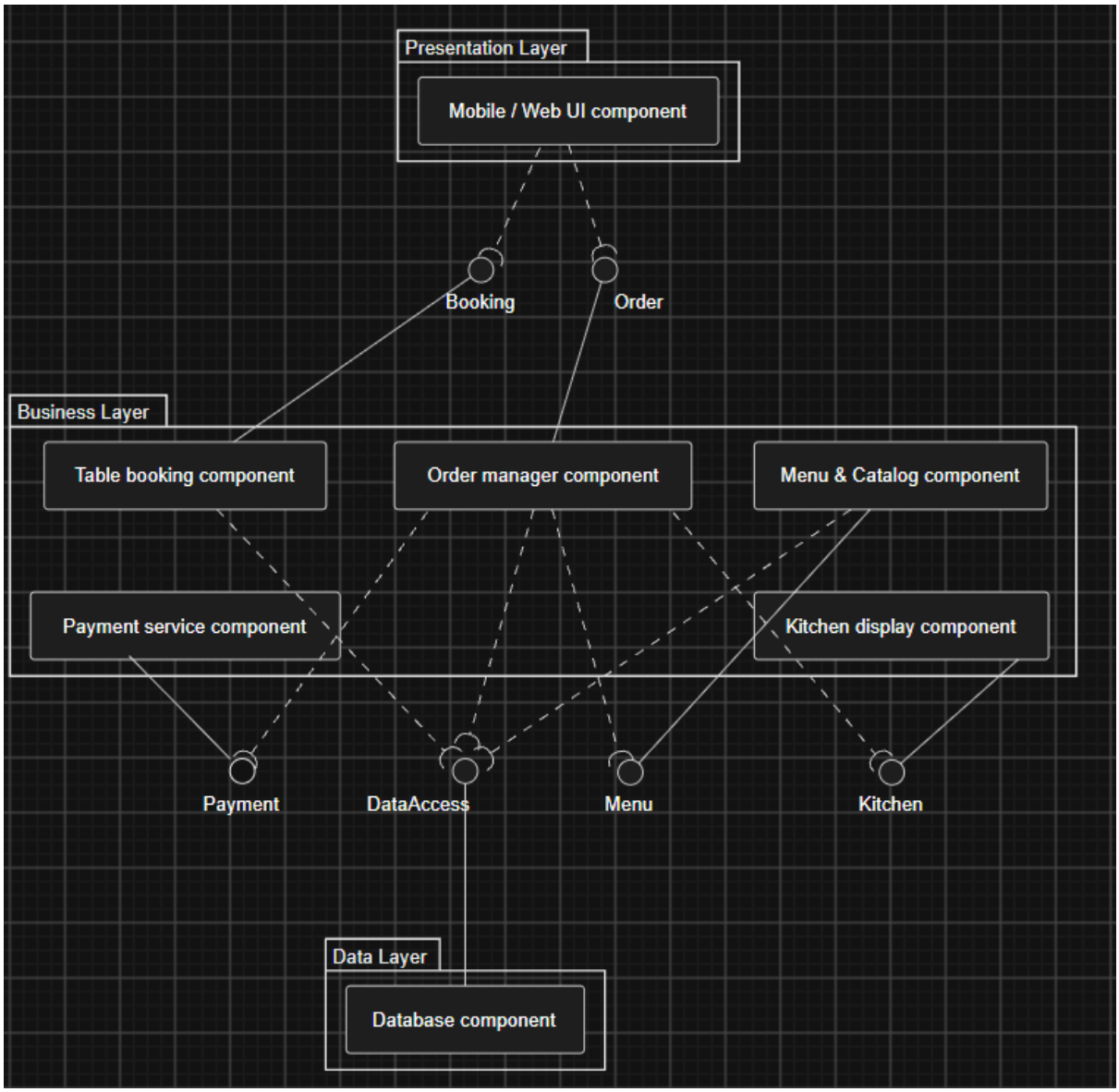
Lab 3: Component Modelling & Architectural Pattern Selection
Name- Rithish PM, SRN- PES2UG24CS405

Component diagram-



We chose Layered architecture for the Restaurant Table Booking & Pre-Ordering App System.

Architectural choice:
We selected the Layered Architecture (3-Tier Architecture consisting of Presentation Layer, Business Layer and Data Layer).

Two Reasons:
1. Clear separation of concerns & modularity:
A table booking and meal pre-ordering application consists of distinct business sub-domains—managing table reservations, cataloging dynamic menu items, computing order bills, and interfacing with payment gateways. Layered architecture cleanly encapsulates these operations within dedicated business components, ensuring changes to kitchen menu schedules or floor layouts do not affect the customer-facing mobile UI or the underlying database schema.

2. Avoidance of operational complexity & data inconsistencies:
Unlike microservices, which introduce high deployment overhead, distributed transaction challenges, and eventual consistency delays, a layered architecture ensures strong ACID consistency during synchronous table locking and meal checkout. This prevents critical scenario errors such as double-booking the same table or confirming an order without reserved seating.

Security Advantage:
Isolation of sensitive data & tier protection: The presentation layer (customer mobile/web app) is completely decoupled from the data persistence layer. User requests cannot directly query or access the database. All payment processing requests and sensitive customer reservation details are strictly mediated through the business layer (specifically the payment service component), allowing centralized implementation of input validation, role-based access control (RBAC), and PCI-DSS compliance standards while defending against direct SQL injection attacks.

Performance Benefit:
Low latency & high in-memory throughput: By co-locating core transactional components (order manager, table booking, and menu components) within the same execution environment in the business layer, the system performs table-availability validations and pre-order pricing calculations via efficient in-memory calls rather than incurring repeated cross-network serialization, deserialization, and HTTP/gRPC overhead common in distributed architectures.